

Agente Rafa para Connect-4

TBOPI con memoria global aprendida por self-play

Rafael Salcedo

Idea principal

El agente final, **RafaImproved**, usa *trial-based online policy improvement* (TBOPI). En cada turno ejecuta una búsqueda local con UCB desde el estado actual; esa búsqueda empieza con una Q-table global aprendida por self-play. Se distingue de otros enfoques del grupo porque no depende solo de una heurística fija ni solo de búsqueda online: combina aprendizaje offline persistente con mejora online por turno. La memoria no reemplaza el razonamiento online: funciona como un *prior* que guía la exploración hacia acciones útiles.

El entrenamiento asumió que no conocemos la política óptima. Primero se entrenó **RafaQ** con 50,000 partidas y 24 inner rollouts por estado. Luego se entrenó **RafaImproved** con otras 50,000 partidas y 8 rollouts por estado, pero de mejor calidad por ser heurísticos: en las simulaciones internas intenta ganar si puede, bloquear amenazas inmediatas, evitar jugadas que entregan victoria al rival y preferir columnas centrales. La memoria final tiene aproximadamente **1.99M estados** y **2.17M pares estado-acción**. Código final: [groups/Rafa/policy.py](#) en el branch **Policy-Rafael**.

Diferencia frente al grupo. Fermín usa MCTS online puro con UCB1: construye un árbol durante el turno y simula rollouts aleatorios, sin memoria persistente entre partidas. Tomás usa First-Visit Monte Carlo entrenado en `mount()` contra un rival aleatorio, con Q-values que guían la acción después de revisar ganar/bloquear. Rafa queda entre ambos enfoques: conserva memoria persistente aprendida por self-play y aún así mejora cada decisión online con UCB local.

Versiones comparadas

Para no presentar el agente como una caja negra, comparé tres versiones internas, todas implementadas en el mismo archivo de código. **RafaBase** usa TBOPI local sin memoria. **RafaQ** agrega Q-values globales como prior, pero mantiene rollouts aleatorios. **RafaImproved** es la versión final: usa memoria global, rollouts heurísticos y selección robusta por visitas/valor.

La Figura 1 muestra mejoras incrementales: el porcentaje de victorias en la ablation local sube de **35 %** en RafaBase a **50 %** en RafaQ y **60 %** en RafaImproved. Esto sugiere que la memoria global aporta, pero que la mayor mejora aparece cuando también se mejora la calidad de los inner trials.

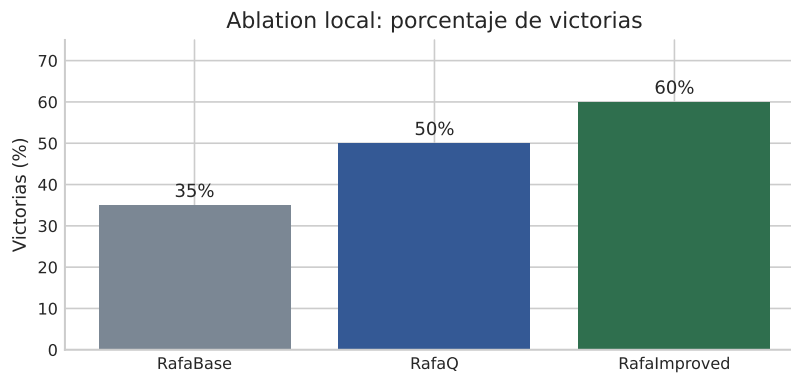
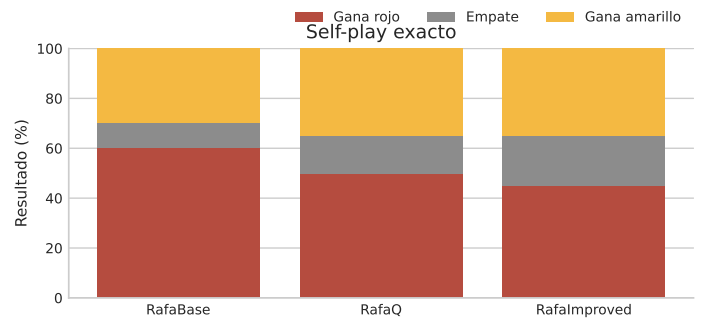
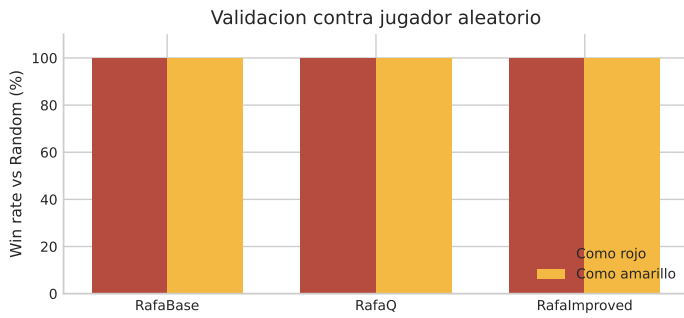


Figura 1. Porcentaje de victorias en comparación local entre versiones.

Validación mínima

El desempeño se midió en función del oponente, no como un único número global. Contra el jugador aleatorio, las tres versiones ganaron el **100 %** de 20 partidas como rojo y 20 como amarillo, sin derrotas. Esto cumple el prerrequisito del reto. También se evaluó el autodesempeño: cada versión jugando contra una copia exacta de sí misma. En RafaImproved aparecen más empates (20 %), lo cual indica una política más estable cuando ambos lados aplican la misma estrategia.



Random por versión y color.

Self-play exacto.

Figura 2. Validación mínima: Random y self-play exacto.

Sweep y uso de memoria

Para analizar configuración y recursos, se varió `total_time` (presupuesto online por partida) y `global_prior_visits` (peso de la memoria en UCB). El mejor punto observado contra RafaQ fue **prior=10** y **2s**, con 100 % de victorias en esa muestra. La conclusión no es que más tiempo siempre gane, sino que el peso de la memoria debe calibrarse: muy bajo ignora experiencia y muy alto puede sobreconfiar en estimaciones ruidosas.

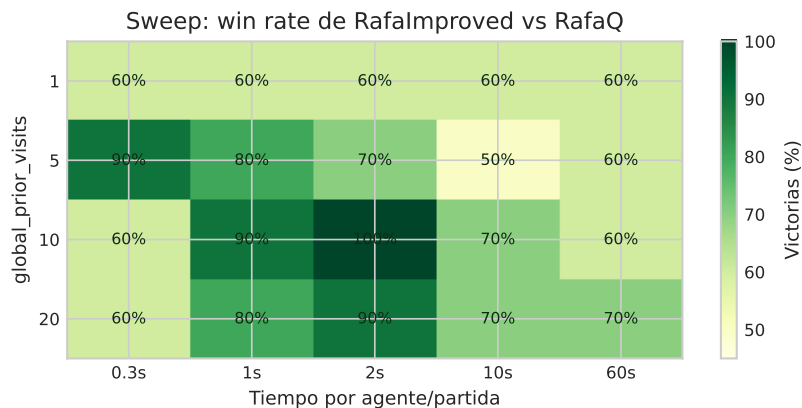


Figura 3. Sweep de RafalImproved vs RafaQ en porcentaje de victorias.

La cobertura aproximada de estados vistos cae fuertemente a medida que avanza la partida. Esto muestra que, aunque la Q-table es grande, muchos estados de medio/final de partida siguen dependiendo de búsqueda online.

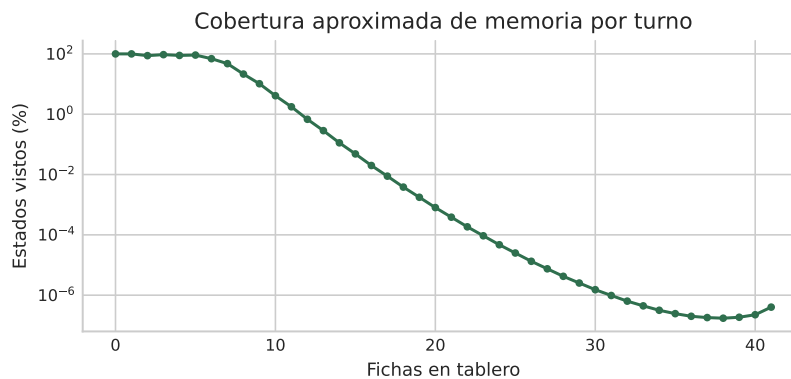


Figura 4. Porcentaje aproximado de estados vistos por número de fichas.

Conclusiones y mejoras

RafalImproved supera a RafaQ porque no solo recuerda valores, sino que hace rollouts de mayor calidad y decide de forma más robusta. Sin embargo, el análisis muestra que el límite principal no es Random, sino la cobertura: en partidas avanzadas hay demasiados estados posibles y la memoria empieza a ser dispersa. Cuando eso ocurre, el agente vuelve a depender casi totalmente de la búsqueda online del turno.

La mejora más directa es **canonicalizar simetría horizontal**. En Connect-4, muchas posiciones son equivalentes a su espejo; hoy se guardan como estados distintos, duplicando aprendizaje. También conviene entrenar **posiciones de medio/final de partida**: en lugar de iniciar todos los self-play desde tablero vacío, se pueden muestrear estados intermedios y reforzar justo las profundidades donde la cobertura cae. Finalmente, se deberían guardar **trayectorias compactas** en evaluación para medir el hit-rate real: no solo cuántos estados existen en la Q-table, sino cuántas decisiones de torneo realmente encuentran memoria útil.